

Pruebas unitarias en Python

Las **pruebas unitarias** son fragmentos de código que verifican el funcionamiento correcto de funciones o métodos específicos de un programa. Actúan como **controles de calidad** que aseguran que cada componente del software cumpla su propósito, al igual que se prueban vigas y columnas en la construcción de un rascacielos.

Importancia de las pruebas unitarias

1. Detección temprana de errores

- Permiten identificar fallos en etapas iniciales, lo que los hace más fáciles y baratos de corregir.
- Funcionan como un sistema de alerta temprana que evita que pequeños errores escalen.

2. Código modular y limpio

- Fomentan la escritura de funciones más específicas, independientes y fáciles de mantener.
- Facilitan modificaciones futuras sin introducir nuevos errores.

3. Seguridad en la refactorización

- Sirven como “arnés de seguridad”.
- Antes y después de modificar el código, las pruebas aseguran que la funcionalidad original se mantenga intacta y que no aparezcan regresiones.

4. Casos prácticos

- **Carrito de compras:** Validar cálculos con impuestos, descuentos y envíos.
- **Formulario de registro:** Comprobar correos válidos, contraseñas seguras y rangos de edad.
- **Nómina:** Asegurar cálculos correctos de horas extras y pagos.

Ventajas adicionales

- **Entornos colaborativos:** Evitan que los cambios de un desarrollador afecten negativamente otras partes del sistema.
- **Proyectos complejos:** Aseguran que cada componente funcione como parte de una red interconectada.
- **Autodocumentación:** Las pruebas describen de manera práctica el propósito y comportamiento del código.

Conclusión

Integrar pruebas unitarias en el desarrollo en Python es una **inversión estratégica**.

- Mejora la calidad y fiabilidad del software.
- Reduce riesgos al hacer cambios.
- Ahorra tiempo de depuración.
- Da como resultado un código más limpio, mantenible y robusto.

Pregunta

Estás trabajando en un proyecto colaborativo de Python con una gran base de código. Varios desarrolladores están contribuyendo con nuevas características y modificando el código existente. En este escenario, ¿cuál de las siguientes opciones describe **mejor** la función principal de las pruebas unitarias? Seleccione la mejor respuesta.

- ☒ Actuar como red de seguridad, permitiendo a los desarrolladores refactorizar el código con confianza sin temor a romper la funcionalidad existente
- ☐ Generación automática de nuevas características y funcionalidades para la aplicación
- ☐ Garantizar que el aspecto visual de la aplicación sea estéticamente agradable y coherente
- ☐ Documentar los requisitos y especificaciones del proyecto para futuras referencias

✔ **Correcto**

Correcto Las pruebas unitarias verifican que los cambios en el código no introducen regresiones o efectos secundarios inesperados, lo que da a los desarrolladores la confianza necesaria para refactorizar y mejorar el código.

Pruebas unitarias como control de calidad del software

Las **pruebas unitarias** en Python consisten en evaluar las partes más pequeñas del código —funciones o métodos— de manera aislada, asegurando que cada componente funcione correctamente antes de integrar el sistema completo.

Analogía con una máquina

- Probar cada **función** es como inspeccionar cada engranaje o circuito antes de montar toda la máquina.
- Garantiza que cada pieza resista condiciones normales, límites de diseño y errores previstos.
- Igual que en mecánica, las pruebas unitarias someten el código a **pruebas de estrés** con entradas típicas, límites y casos inválidos.

Propósito y beneficios

1. Sistema de alerta temprana

- Detectan errores inmediatamente después de hacer cambios.
- Previenen que los fallos lleguen a producción (ej. nómina con deducciones mal calculadas).

2. Control de calidad automatizado

- Cada ejecución de pruebas valida el comportamiento esperado de las funciones.
- Una prueba fallida indica con precisión dónde está el error.

3. Confianza para refactorizar

- Funcionan como una red de seguridad durante la introducción de nuevas funciones o cambios de código.
- Garantizan que modificar una parte no rompa otra.

4. Proceso sistemático

- Diseñar casos de prueba → ejecutar pruebas → analizar fallos → corregir código → repetir hasta aprobar.

Conclusión

Las pruebas unitarias no solo buscan **aprobar pruebas**, sino:

- Explorar cómo el código responde en diversos escenarios.
- Prevenir errores costosos.
- Asegurar que el software sea **fiable, estable y evolutivo**.

Del mismo modo que probar cada engranaje garantiza una máquina robusta, un código cubierto por pruebas unitarias da lugar a un software sólido y duradero.

Pregunta

Se te ha encargado optimizar una función compleja de Python que realiza cálculos computacionalmente intensivos. Quieres asegurarte de que tus optimizaciones no introducen inadvertidamente errores o regresiones. ¿Cuál de los siguientes enfoques sería el MÁS beneficioso en este escenario? Seleccione la mejor respuesta.

- ☐ Implemente sentencias de impresión dentro de la función para realizar un seguimiento de los valores intermedios y asegurarse de que se ajustan a sus expectativas.
- ☐ Confiar únicamente en las pruebas manuales y la inspección visual del código para verificar su corrección tras la optimización.
- ☐ Confíe en los comentarios de los usuarios después de desplegar la función optimizada en producción para identificar posibles problemas.
- ☒ Escribir pruebas unitarias completas para la función antes y después de la optimización, comparando los resultados para garantizar la precisión e identificar cualquier regresión.

✓ Correcto

Correcto Las pruebas unitarias proporcionan una red de seguridad para la refactorización. Permiten comparar la salida de las funciones originales y optimizadas para garantizar que producen los mismos resultados y detectar cualquier efecto secundario no deseado.

Fundamentos de las pruebas unitarias en Python

Las **pruebas unitarias** son una práctica clave para garantizar la **calidad y fiabilidad del código**. Consisten en verificar de manera aislada las unidades más pequeñas del programa (funciones o clases) para asegurarse de que se comportan como se espera.

Conceptos principales

- **Aserciones:** verifican si el resultado del código coincide con lo esperado.
- **Casos de prueba:** conjunto de aserciones que prueban una función o clase en distintos escenarios.
- **Conjunto de pruebas:** colección de varios casos que abarcan diferentes partes del sistema.
- **Ejecutor de pruebas:** herramienta que corre todos los casos y reporta qué pasó o falló.

Beneficios

1. **Detección temprana de errores:** previene fallos grandes a partir de pequeños errores.
2. **Confianza en el código:** permite refactorizar sin miedo, gracias a la red de seguridad de las pruebas.
3. **Mejora del diseño:** fomenta un código modular y fácil de mantener.
4. **Documentación viva:** los casos de prueba muestran cómo usar el código y facilitan su comprensión.

Ciclo de las pruebas unitarias

1. **Escribir código** (crear función o clase).
2. **Escribir pruebas** (definir aserciones y escenarios).
3. **Ejecutar pruebas** (el ejecutor valida los resultados).
4. **Corregir y repetir** hasta que todo pase correctamente.

Este ciclo es continuo y se integra en el flujo de desarrollo.

Ejemplo práctico en Python con **pytest**

Código para probar una función que calcula un descuento:

```
import pytest

def calculate_discount(price, percentage):
    return price - (price * percentage / 100)

class TestDiscountCalculation:
    def test_ten_percent_discount(self):
        result = calculate_discount(100, 10)
        assert result == 90  # Assertion: se espera 90
```

```
def test_invalid_input(self):  
    with pytest.raises(TypeError):  
        calculate_discount("100", 10) # Entrada incorrecta
```

Conclusión

Las pruebas unitarias no son un paso opcional, sino una **habilidad fundamental** para crear software robusto. Ayudan a detectar errores temprano, facilitan la refactorización y mejoran la calidad del diseño. Adoptarlas como parte del proceso de desarrollo convierte cada prueba superada en una **pequeña victoria hacia proyectos más confiables**.

Cuestionario

1. ¿Cuál es el objetivo principal de las pruebas unitarias en el desarrollo de software? Seleccione la mejor respuesta.

- ☐ Para garantizar el rendimiento del software bajo cargas pesadas
- ☐ Para verificar que el software cumple los requisitos del usuario
- ☐ Integrar diferentes módulos del software
- ☒ Identificar y corregir errores en una fase temprana del ciclo de desarrollo

✓ **Correcto**

Correcto Las pruebas unitarias ayudan a detectar y resolver errores en una fase temprana, lo que garantiza una base de código más estable.

2. ¿Cuál de las siguientes afirmaciones es cierta sobre las aserciones en las pruebas unitarias? Seleccione todas las que correspondan.

- ☐ Las aserciones pueden corregir automáticamente errores en el código
- ☐ Las aserciones pueden sustituir a las pruebas de integración
- ☒ Las aserciones son un componente clave de la redacción de casos de prueba

✓ **Correcto**

Correcto Las aserciones son esenciales en los casos de prueba para validar el comportamiento del código.

- ☒ Las aserciones ayudan a verificar que el código se comporta como se espera

✓ **Correcto**

Correcto Las aserciones se utilizan para garantizar que el código produce los resultados esperados.

3. Usted está escribiendo una prueba unitaria para una función llamada `calculate_discount(price, is_member)`. ¿Cuál de los siguientes sería el enfoque más apropiado para esta prueba unitaria? Seleccione la mejor respuesta.

- ☒ Para comprobar que la función devuelve el importe de descuento correcto para diferentes valores de entrada de `price` y `is_member`.
- ☐ Integrar diferentes módulos del software
- ☐ Para verificar que el software cumple los requisitos del usuario
- ☐ Para garantizar el rendimiento del software bajo cargas pesadas

✓ **Correcto**

Correcto Esto prueba correctamente la funcionalidad de la función.

4. Un desarrollador es nuevo en el mundo de las pruebas y le pregunta sobre la práctica de las pruebas unitarias. ¿Cuál sería la mejor manera de describirle las pruebas unitarias? Seleccione la mejor respuesta.

- ☐ Las pruebas unitarias sólo se realizan al final del proceso de desarrollo para garantizar que todo funciona según lo esperado.
- ☐ Las pruebas unitarias son un método para probar toda la aplicación a la vez y asegurarse de que todo funciona correctamente.
- ☐ Las pruebas unitarias son lo mismo que las pruebas de aceptación del usuario, en las que los usuarios finales prueban la aplicación.
- ☒ Las pruebas unitarias consisten en probar componentes o funciones individuales del código de forma aislada para verificar su correcto comportamiento.

✓ **Correcto**

Correcto Las pruebas unitarias se centran en probar unidades individuales de código.

5. Está explicando las pruebas unitarias a una parte interesada no técnica. ¿Cuál de estas descripciones es la más adecuada? Seleccione la mejor respuesta.

- ☐ Las pruebas unitarias son el último paso antes de poner el software a disposición de los usuarios.
- ☒ Las pruebas unitarias son como comprobar las piezas individuales del motor de un coche antes de montarlo entero.
- ☐ Las pruebas unitarias sólo son necesarias en proyectos de software grandes y complejos.
- ☐ Las pruebas unitarias las realizan los usuarios finales para asegurarse de que el software satisface sus necesidades.

✓ **Correcto**

Correcto. Es una buena analogía Destaca la idea de probar los componentes individuales antes de integrarlos.

Introducción a pytest en Python

(Código importante)

¿Qué es pytest?

- Es un **framework de pruebas** para Python.
- Permite escribir pruebas **simples, legibles y escalables**, desde unitarias hasta de integración complejas.

- Actúa como una **red de seguridad**: detecta errores temprano, ahorrando tiempo y esfuerzo.

Instalación y uso básico

Instalar:

```
pip install pytest
```

1.

Crear función en **calculator.py**:

```
def add(a, b):  
    return a + b
```

2.

Crear prueba en **test_calculator.py**:

```
def test_add():  
    assert add(2, 3) == 5
```

3.

Ejecutar pruebas desde la terminal:

```
pytest
```

4.

Si la función es incorrecta, pytest muestra el **error y la causa exacta**.

Características clave

- **Aserciones**: verifican que el resultado sea el esperado.
- **Parametrización**: permite probar una función con **múltiples entradas y salidas** en un solo test.
- **Detección automática**: pytest reconoce archivos que empiezan con **test_** y funciones que siguen la misma convención.

Accesorios (Fixtures)

- Son funciones reutilizables que preparan datos o estados para las pruebas.
- Se definen con `@pytest.fixture`.
- Se pueden usar en varias pruebas, y su alcance puede ajustarse:
 - **función (default)** → se ejecuta en cada prueba.
 - **módulo, clase o sesión** → permanece disponible más tiempo.

Ejemplo simple de fixture:

```
import pytest

@pytest.fixture
def sample_data():
    return [10, 20, 30, 40]

def test_sum(sample_data):
    assert sum(sample_data) == 100

def test_max(sample_data):
    assert max(sample_data) == 40
```

Ejemplo avanzado: un fixture que transforma datos y muestra mensajes de inicialización para depuración.

Conclusión

- pytest ofrece **sintaxis limpia, flexibilidad y potencia** para gestionar pruebas desde las más simples hasta escenarios complejos.
- Con **aserciones, parametrización y fixtures**, garantiza un flujo de pruebas confiable.
- Las pruebas son un **proceso continuo**: deben escribirse y ejecutarse mientras se desarrolla el código para asegurar estabilidad y calidad.

Trucos y consejos pytest

1. Dispositivos (Fixtures)

- Automatizan la **configuración y limpieza** de pruebas.
- Eliminan código repetitivo y garantizan un **entorno controlado**.
- Usos comunes:
 - Generación de datos de prueba reutilizables.
 - Preparación de condiciones previas (bases de datos, APIs simuladas).
 - Limpieza posterior (cerrar conexiones, eliminar archivos).
- Ejemplo: fixture que gestiona el inicio y cierre de sesión en pruebas de aplicaciones web.

2. Parametrización

- Permite ejecutar la misma prueba con **múltiples entradas y salidas**.
- Facilita probar:
 - Valores límite.
 - Entradas no válidas.
 - Casos extremos.
- Aumenta la **cobertura** y la **confiabilidad** del código.
- Ejemplo: validar un formulario de registro con correos, contraseñas y edades diferentes.

3. Organización de pruebas

- Las pruebas pueden estructurarse en **módulos, clases o directorios**, reflejando la arquitectura del código.
- Beneficios:
 - Navegación clara.
 - Escalabilidad y mantenimiento más sencillo.
 - Facilita la incorporación de nuevos desarrolladores.
- Marcadores útiles:
 - `@pytest.mark.skip` → omitir pruebas no válidas.
 - `@pytest.mark.xfail` → marcar pruebas que se espera que fallen.

4. Buenas prácticas y extras

- Usar **nombres claros e informativos** para las funciones de prueba.
- Evitar dependencias entre pruebas → deben poder ejecutarse en cualquier orden.
- Aprovechar **plugins de pytest** (cobertura, pruebas de bases de datos, simulación, etc.).
- Combinar **fixtures + parametrización** para pruebas más potentes.

Conclusión

Dominar estas técnicas avanzadas de pytest (fixtures, parametrización y organización) permite:

- Reducir redundancia.
- Mejorar la cobertura.
- Detectar errores complejos.
- Refactorizar con confianza.

Con ello, se construye un **proceso de pruebas más eficiente, robusto y mantenible**, ideal para proyectos Python en crecimiento.

Pregunta

Está desarrollando una función de biblioteca de Python que realiza cálculos complejos basados en varios parámetros de entrada. Quiere asegurarse de que esta función maneja correctamente múltiples escenarios de entrada y casos extremos. ¿Qué función avanzada de pytest es la más adecuada para lograr este objetivo? Seleccione la mejor respuesta.

- ☐ Accesorios
- ☐ Organización de pruebas
- ☒ Parametrización
- ☐ Afirmaciones

✔ **Correcto**

Correcto La parametrización le permite ejecutar la misma función de prueba con diferentes valores de entrada, probando eficientemente varios escenarios y casos extremos sin escribir funciones de prueba separadas.

pytest y sus características principales

¿Qué es pytest?

- Es un **framework de pruebas para Python**, potente y fácil de usar.
- Principios clave: **simplicidad** y **flexibilidad**.
- Permite escribir pruebas compactas y legibles, desde casos simples hasta escenarios complejos.
- Funciona como una “red de seguridad” que detecta errores y problemas antes de llegar a producción.

¿Por qué usar pytest?

- Sintaxis **intuitiva** y poco código repetitivo.
- Se integra sin fricción en el flujo de desarrollo.
- Tiene un **ecosistema amplio de plugins** que amplían sus capacidades.

Conceptos y características principales

1. Funciones de prueba

- Son funciones Python normales cuyo nombre empieza con `test_`.
- Contienen **aserciones** que validan el comportamiento esperado del código.
- Si una falla, pytest señala el error.

2. Fixtures (accesorios)

- Gestionan **configuración y limpieza** de recursos.
- Aseguran entornos de prueba aislados y reutilizables.
- Ejemplos: conexión temporal a una base de datos, simulación de API externa.

3. Parametrización

- Permite ejecutar la **misma prueba con diferentes entradas**.
- Garantiza que el código maneje múltiples escenarios de forma correcta.

4. Marcadores

- Etiquetas para **clasificar pruebas**.
- Posibilitan ejecutar grupos específicos de pruebas.
- Ejemplo: marcar las pruebas que requieren servicios externos para poder omitirlas en ciertas ejecuciones.

5. Plugins

- pytest es altamente **extensible**.
- Existen plugins para pruebas de web, bases de datos, código asíncrono, entre otros.
- Facilitan y optimizan el proceso de pruebas.

En resumen: **pytest** es un marco versátil que simplifica la escritura, organización y ejecución de pruebas en Python, con soporte para **fixtures**, **parametrización**, **marcadores** y **plugins**, ofreciendo potencia y flexibilidad para cualquier tipo de proyecto.

Escenarios reales con pytest

Escenario 1: Aplicaciones Web (Flask)

- **Objetivo:** Garantizar que el registro, login y demás interacciones funcionen correctamente.
- **pytest + pytest-flask** permiten:
 - **Simular interacciones de usuario** (formularios, rutas, carga de archivos).
 - **Verificar respuestas** (códigos de estado, contenido en HTML/JSON).
 - **Comprobar integridad de la base de datos** tras operaciones (insertar, actualizar, eliminar).

Código de ejemplo:

```
import pytest
from your_flask_app import app  # Import your Flask app

@pytest.fixture
def client():
    """Create a test client for interacting with the Flask app."""
    app.config['TESTING'] = True  # Enable testing mode
    with app.test_client() as client:
        yield client

def test_user_registration(client):
    """Simulate a user registration and check the response."""
    data = {'username': 'testuser', 'email': 'testuser@example.com',
            'password': 'testpassword'}
    response = client.post('/register', data=data)

    assert response.status_code == 302  # Expect a redirect after
    successful registration
    # Further checks on database or session data can be added here

def test_login(client):
    """Simulate a user login and verify the response."""
```

```
data = {'username': 'existinguser', 'password':
'correctpassword'}
response = client.post('/login', data=data)

assert response.status_code == 200 # Expect a successful login
assert b'Welcome, existinguser' in response.data # Check if
welcome message is present
```

Escenario 2: Ciencia de Datos

- **Objetivo:** Asegurar que la limpieza de datos y modelos ML funcionan correctamente.
- pytest permite:
 - **Validar transformaciones de datos** (manejo de valores faltantes, outliers, formato).
 - **Verificar predicciones del modelo** comparándolas con la verdad conocida.
 - **Evaluar rendimiento** usando métricas como exactitud, precisión, recall y F1.

Código de ejemplo:

```
import pytest
import pandas as pd
from your_data_science_project import clean_data, train_model,
predict

# Sample test data
@pytest.fixture
def test_data():
    data = {'feature1': [1, 2, 3, None], 'feature2': ['A', 'B', 'C',
'D']}
    return pd.DataFrame(data)

def test_data_cleaning(test_data):
    """Verify if data cleaning handles missing values correctly."""
    cleaned_data = clean_data(test_data)
    assert cleaned_data['feature1'].isnull().sum() == 0 # Check if
missing values are filled
```



```
def test_model_predictions():
    """Check if model predictions match expected outcomes."""
    X_test = ... # Load your test data
    y_test = ... # Load corresponding ground truth labels
    model = train_model(...)
    predictions = predict(model, X_test)
    accuracy = (predictions == y_test).mean()
    assert accuracy > 0.8 # Set your desired accuracy threshold

def test_model_performance():
    """Evaluate model performance using relevant metrics."""
    # ... Load your evaluation data and calculate metrics (e.g.,
    precision, recall, F1-score)
    # Assert that the metrics meet your expectations.
```

Conclusiones

- Las pruebas **no son opcionales**, son una necesidad:
 - Detectan errores temprano.
 - Funcionan como documentación viva del comportamiento esperado.
- Aunque Python incluye `unittest`, **pytest es más simple y potente** (fixtures, parametrización, plugins).
- Ya sea en aplicaciones web o ciencia de datos, pytest es un **compañero confiable para asegurar calidad, fiabilidad y mantenibilidad** del código.

Reto de codificación: Comprobación simple de la pertenencia a una lista

```
import pytest
```

```
def test_contains_five():
    """
    Tests the 'contains_five' function.
    """
    # Example list for testing
    my_list = [1, 3, 5, 7, 9]
```

```
assert contains_five(my_list) == True # nos quiere decir si el 5
esta en la lista
```

Conclusión

Una función de prueba bien escrita hace más que ejecutar código; está verificando funcionalidad. En este caso, verifica que la función `contains_five`, cuando se le da una lista específica (como `my_list`), identifica correctamente la presencia del número 5. Si la prueba pasa, `contains_five` se comporta como se espera, y si falla, identifica un problema en `contains_five`.

Si estuviera escribiendo un conjunto de pruebas exhaustivo, consideraría muchos escenarios diferentes para la función `contains_five`. Además de comprobar una lista en la que 5 está presente, querría crear pruebas para casos en los que 5 no está en la lista, una lista vacía (`[]`), una lista que sólo contiene el número 5 y una lista con múltiples apariciones de 5. Diversas pruebas ayudan a garantizar que su función maneja todas las entradas correctamente.

También hay formas de comprobar la eficacia de su conjunto de pruebas (conocidas como pruebas de mutación), pero esa es una lección que va más allá de este certificado.

Organización y estructura de pruebas en pytest

Idea principal

- Organizar las pruebas en pytest es tan importante como seguir una receta en la cocina: garantiza claridad, mantenimiento y eficacia.
- Un conjunto de pruebas bien estructurado permite localizar errores fácilmente, refactorizar con confianza y mantener un flujo de desarrollo estable.

Beneficios de la organización de pruebas

- **Claridad:** pruebas con nombres claros y objetivos definidos.
- **Mantenimiento:** localizar y modificar pruebas específicas sin dificultad.
- **Eficiencia:** depuración más rápida y reducción de errores ocultos.

Principios clave de organización

1. Modularidad

- Cada prueba debe enfocarse en una parte específica del código.
- Permite detectar errores puntuales y facilita la refactorización.

Ejemplo en código:

```
import pytest
```

```
class Calculator:
```

```
    def add(self, x, y):
```

```
        return x + y
```

```
    def subtract(self, x, y):
```

```
        return x - y
```

```
    def multiply(self, x, y):
```

```
        return x * y
```

```
def divide(self, x, y):
    if y == 0:
        raise ValueError("Division by zero!")
    return x / y


class CalculatorTest:
    @pytest.fixture
    def calculator(self):
        from calculator import Calculator
        return Calculator()

    def test_add(self, calculator):
        result = calculator.add(2, 3)
        assert result == 5

    def test_subtract(self, calculator):
        result = calculator.subtract(5, 2)
        assert result == 3

    def test_multiply(self, calculator):
        result = calculator.multiply(4, 3)
        assert result == 12

    def test_divide(self, calculator):
```

```
result = self.calculator.divide(10, 2)

assert result == 5
```

```
def test_divide_by_zero(self, calculator):

    with pytest.raises(ValueError):

        calculator.divide(5, 0)
```

2.

- La clase `Calculator` define operaciones básicas.
- La clase `CalculatorTest` contiene pruebas independientes para cada operación.
- El fixture asegura una instancia limpia de `Calculator` en cada test.

2. Convenciones de nomenclatura

- Usar nombres descriptivos: `test_calculate_total_price` es más útil que `test_1`.
- Facilita la navegación y comprensión de las pruebas.

3. Fixtures

- Permiten configurar y desmontar entornos de prueba compartidos.
- Reducen redundancia y garantizan coherencia entre pruebas.

4. Marcadores

- Clasifican pruebas según características (`@pytest.mark.slow`, personalizados, etc.).
- Permiten ejecutar subconjuntos específicos para optimizar tiempo y recursos.

5. Parametrización

- Ejecuta la misma prueba con diferentes entradas.

- Asegura que el código funcione bajo múltiples escenarios y ayuda a descubrir errores ocultos.

Conclusión:

Una buena organización en pytest (modularidad, nombres claros, fixtures, marcadores y parametrización) convierte el conjunto de pruebas en una herramienta poderosa y mantenible que asegura calidad, claridad y confianza en el desarrollo.

Escenario real con pytest

Contexto

- Escenario: desarrollo de una **plataforma de comercio electrónico**.
- Se aplican principios de **pytest** (modularidad, fixtures, nomenclatura, etc.) para mantener las pruebas organizadas y efectivas.
- Un conjunto bien estructurado asegura calidad, facilita la depuración y agiliza el trabajo en equipo.

Funciones críticas probadas

1. Productos y carrito

- `test_add_to_cart_successful`: verifica que los productos se agreguen correctamente.
- `test_calculate_total_price_accuracy`: comprueba cálculo de precios totales con impuestos y envío.
- `test_apply_discount_correctly`: asegura aplicación correcta de descuentos.
- `test_checkout_successful_payment`: simula proceso de pago con pasarela mockeada.

2. Interacciones con usuarios

- `test_create_new_user_account_valid_data`: valida creación de cuentas con datos correctos.
- `test_login_successful_credentials`: verifica login exitoso con credenciales válidas.
- `test_update_user_information_changes_saved`: confirma que los cambios en el perfil se guardan.

Código de ejemplo simplificado

```
import pytest
```

```
class Product:
```

```
    def __init__(self, name, price, quantity=1):
```

```
        self.name = name
```

```
        self.price = price
```

```
        self.quantity = quantity
```

```
class Cart:
```

```
    def __init__(self):
```

```
        self.items = []
```

```
    def add_to_cart(self, product):
```

```
        self.items.append(product)
```

```
    def calculate_total_price(self, tax_rate=0.0, shipping_fee=0.0):
```

```
        subtotal = sum(item.price * item.quantity for item in  
self.items)
```

```
tax = subtotal * tax_rate

total = subtotal + tax + shipping_fee

return total
```

```
def apply_discount(self, discount_percentage):

    for item in self.items:

        item.price *= (1 - discount_percentage / 100)


def checkout(self, payment_gateway):

    total_price = self.calculate_total_price()

    if payment_gateway.process_payment(total_price):

        return True

    else:

        return False
```

```
class TestProductFunctions:

    def test_add_to_cart_successful(self):

        cart = Cart()

        product = Product("Widget", 10.0)

        cart.add_to_cart(product)

        assert len(cart.items) == 1

        assert cart.items[0] == product


    def test_calculate_total_price_accuracy(self):
```



```

    cart = Cart()

    cart.add_to_cart(Product("Widget A", 15.0))

    cart.add_to_cart(Product("Widget B", 20.0, 2)) # 2 unidades

    total = cart.calculate_total_price(tax_rate=0.07,
shipping_fee=5.0)

    expected_total = (15.0 + 20.0 * 2) * 1.07 + 5.0

    assert total == expected_total


def test_apply_discount_correctly(self):

    cart = Cart()

    cart.add_to_cart(Product("Discounted Item", 50.0))

    cart.apply_discount(20) # 20% descuento

    assert cart.items[0].price == 40


def test_checkout_successful_payment(self, mocker):

    mock_payment_gateway = mocker.Mock()

    mock_payment_gateway.process_payment.return_value = True


    cart = Cart()

    cart.add_to_cart(Product("Something", 100.0))

    result = cart.checkout(mock_payment_gateway)

    assert result is True

```

Buenas prácticas aplicadas

- **Modularidad:** separar pruebas en archivos como `test_product_functions.py` y `test_user_interactions.py`.
- **Nombres descriptivos:** cada función explica claramente su propósito.
- **Fixtures:** pueden crear carritos vacíos (`empty_cart`) o productos de prueba (`test_product`) reutilizables.
- **Organización:** pruebas agrupadas por dominios (productos, usuarios).

Conclusión:

Dedicar tiempo a estructurar las pruebas no es sobrecarga, sino una inversión: mejora la calidad, reduce el tiempo de depuración y facilita la colaboración en proyectos complejos como un e-commerce.

Cuestionario

1. Estás empezando un nuevo proyecto Python y quieres asegurarte de que tus pruebas están bien organizadas desde el principio. ¿Cuál de las siguientes prácticas te ayudará a lograr este objetivo? Selecciona todas las que correspondan.

☒ Coloque sus archivos de prueba en un directorio separado dedicado a las pruebas.

☒ **Correcto**

Correcto Esto ayuda a mantener el proyecto organizado y facilita la localización de las pruebas.

☐ Incluya las dependencias relacionadas con las pruebas en su archivo de requisitos principal.

☒ Nombra tus archivos de prueba con el prefijo test_.

☒ **Correcto**

Correcto Esta convención de nomenclatura permite a los ejecutores de pruebas identificar y ejecutar fácilmente sus pruebas.

☐ Guarde los archivos de prueba en el mismo directorio que el código de implementación.

2. ¿Cuál es la forma recomendada de nombrar los archivos de prueba al organizar el directorio de pruebas? Seleccione la mejor respuesta.

☐ Los archivos de prueba deben llevar el nombre del desarrollador que escribió las pruebas.

☒ Los archivos de prueba deben nombrarse con un prefijo test_ seguido del nombre de la función o módulo que se está probando.

☐ Los archivos de prueba pueden tener nombres arbitrarios siempre que estén en el mismo directorio.

☐ Los archivos de prueba deben nombrarse con el sufijo *_test.

☒ **Correcto**

Correcto Nombrar los archivos de prueba con un prefijo test_ facilita su identificación y ejecución con los ejecutores de pruebas.

3. Estás escribiendo pruebas unitarias para una función de Python llamada `calcular_suma` que toma dos números como entrada y devuelve su suma. Quieres verificar que la llamada a `calcular_suma(4, 6)` produce correctamente la salida 10. ¿Cuál de las siguientes sentencias de Python lo conseguiría? Seleccione la mejor respuesta.

- ☐ `assert calcula_suma(4, 6) != 10`
- ☒ `assert calcular_suma(4, 6) == 10`
- ☐ `assert calcular_suma(4, 6) = 10`
- ☐ `assert calcular_suma(4, 6) es igual a 10`

✔ **Correcto**

Correcto Esta sentencia comprueba correctamente si la función `calcular_suma` devuelve 10 cuando se le dan las entradas 4 y 6.

4. Cuando se utilizan fixtures en un conjunto de pruebas, ¿cuál es la principal ventaja de utilizar el parámetro `scope`? Seleccione la mejor respuesta.

- ☐ Define el orden en que deben ejecutarse las pruebas.
- ☐ Especifica los ajustes de configuración de la prueba para el fixture.
- ☒ Controla el ciclo de vida del accesorio y puede hacer que las pruebas sean más eficaces.
- ☐ Determina el tipo de marco de pruebas que se utilizará.

✔ **Correcto**

Correcto El uso del parámetro de alcance ayuda a gestionar el ciclo de vida del fixture, lo que puede optimizar el rendimiento de la prueba.

5. En el contexto de la parametrización de pruebas en Python, ¿cuáles de las siguientes son razones válidas para utilizar la parametrización de pruebas? Seleccione todas las que correspondan.

- ☐ Hacer que las pruebas dependan unas de otras.
- ☒ Para reducir la necesidad de complejos guiones de prueba.

✔ **Correcto**

Correcto Parametrizar las pruebas simplifica los scripts al eliminar la necesidad de múltiples casos de prueba similares.

- ☐ Para garantizar que las pruebas se ejecutan en una secuencia específica.
- ☒ Para ejecutar la misma prueba con varios conjuntos de entradas de datos.

✔ **Correcto**

Correcto La parametrización de pruebas permite ejecutar la misma lógica de prueba con diferentes conjuntos de datos.

Git: Introducción y conceptos clave

- **Definición:** Git es un **sistema de control de versiones distribuido (DVCS)** que permite a cada desarrollador tener una copia completa del proyecto en su máquina. Esto facilita el trabajo sin conexión, la tolerancia a fallos y la colaboración eficiente.
- **Mecánica de Git:**
 - **Commits:** Son instantáneas del proyecto que registran los cambios con un hash único y un mensaje descriptivo.
 - **Ramas:** Permiten trabajar en nuevas funcionalidades o correcciones de forma aislada, fomentando la experimentación y el desarrollo paralelo.
- **Ventajas principales de Git:**
 - Colaboración fluida entre equipos.
 - Historial completo de versiones con posibilidad de volver atrás.
 - Uso de ramas y fusiones para desarrollo paralelo.
 - Copias de seguridad automáticas al existir múltiples clones del repositorio.
 - Gran comunidad de código abierto que mejora y da soporte a la herramienta.
- **Escenarios de uso real:**
 - **Desarrollo colaborativo:** ramas independientes para cada funcionalidad que luego se integran en la principal.
 - **Corrección de errores:** creación de ramas específicas para solucionar fallos sin afectar al código estable.
 - **Proyectos open source:** forks, ramas para cambios propuestos y pull requests para integrarlos.
- **Retos comunes y soluciones:**
 - **Curva de aprendizaje:** se supera con práctica, tutoriales y GUIs.
 - **Complejidad en grandes proyectos:** se gestiona con buenas prácticas y herramientas de visualización.

- **Errores humanos (commits en ramas equivocadas, fusiones mal hechas):** Git permite recuperarse con comandos como `git reflog` y `git reset`.
- **Seguridad:** evitar exponer información sensible y configurar permisos adecuados.
- **Conclusión:** Git ha transformado el desarrollo de software gracias a su modelo distribuido, su sistema de ramas y su capacidad de colaboración. Aunque presenta desafíos iniciales, sus ventajas superan los inconvenientes, convirtiéndolo en el estándar de facto para el control de versiones.

Aspectos básicos de Git

- **Git es un sistema de control de versiones distribuido (VCS)** que permite a los desarrolladores rastrear cambios, colaborar y experimentar sin riesgos.
 - Proporciona seguridad al permitir volver a versiones anteriores del proyecto.
 - Facilita el trabajo en paralelo y la integración de cambios.
- **Componentes fundamentales:**
 - **Repositorio:** espacio central donde se almacena el código y todo el historial del proyecto. Es como una biblioteca con todas las versiones archivadas.
 - **Confirmación (commit):** una “foto” del proyecto en un momento dado. Registra cambios con un mensaje descriptivo, sirve como punto de control y permite volver atrás si es necesario.
 - **Rama (branch):** línea de desarrollo independiente que permite trabajar en nuevas funciones o correcciones sin afectar al código principal. Luego se pueden fusionar con la rama principal.
- **Instalación e inicialización:**
 - Descargar desde git-scm.com e instalar según el sistema operativo.

Para iniciar un repositorio:

```
git init
```

- Esto crea el directorio oculto `.git` donde se almacena la información del control de versiones.
- **Ejemplo práctico:**
 - Un equipo mantiene estable la rama principal de un sitio web.
 - Crean una nueva rama “*homepage-redesign*” para experimentar con el rediseño de la página de inicio.
 - Hacen varias confirmaciones a medida que avanzan.
 - Finalmente, fusionan la rama con la principal, llevando los cambios a producción sin afectar antes la versión estable.
- **Conclusión:**

Git permite colaboración eficiente, control seguro del historial de cambios, y experimentación sin riesgos. Dominar repositorios, commits y ramas es esencial para un flujo de trabajo ágil y profesional.

Pregunta

Tu equipo está trabajando en una actualización importante de una aplicación web, y varios desarrolladores están implementando nuevas funciones y corrigiendo errores al mismo tiempo. ¿Qué concepto de Git sería el MÁS beneficioso para facilitar este proceso de desarrollo colaborativo y minimizar el riesgo de conflictos?

- ☐ Comprométase
- ☒ Rama
- ☐ Repositorio
- ☐ Instalación

✔ **Correcto**

Entendido Las ramas permiten a los desarrolladores crear espacios de trabajo aislados para sus tareas específicas, minimizando el riesgo de conflictos al fusionar los cambios de nuevo en la base de código principal.

Git como red de seguridad y colaboración

- Git funciona como una **máquina del tiempo para el código**, ya que guarda un historial completo de los cambios y permite volver a cualquier versión anterior.
- Cada **confirmación (commit)** es como una instantánea del proyecto que registra modificaciones junto con un mensaje descriptivo.

- Esto no solo sirve para **corregir errores** al retroceder a versiones estables, sino también para:
 - **Colaborar**: varios desarrolladores pueden trabajar en el mismo proyecto simultáneamente.
 - **Organizar**: los mensajes de commit permiten seguir la evolución del código con claridad.
 - **Experimentar sin riesgos**: las **ramas** permiten probar nuevas ideas o funciones sin afectar el proyecto principal.
- Git garantiza la **integridad del código**, ofrece seguridad frente a errores y fomenta la innovación al dar libertad para explorar sabiendo que siempre se puede volver atrás.

Git para principiantes

- Git es un sistema de **control de versiones** que permite registrar cambios en el código, volver a versiones anteriores y trabajar en ramas independientes.
- Funciona como una **máquina del tiempo** para tu código y facilita la colaboración entre varios desarrolladores.
- Git ayuda a experimentar sin miedo a romper la funcionalidad existente, ya que puedes **revertir cambios o crear ramas para probar nuevas ideas**.

Importancia de Git

- Permite **controlar versiones**, rastrear cambios y comparar estados del código.
- Facilita la **colaboración remota**, integrando trabajo de varios desarrolladores.
- Es el **estándar de la industria**, y dominarlo mejora la productividad y el valor profesional.

Conceptos clave

- **Repositorio**: Carpeta que contiene todo el historial de tu proyecto.
- **Commit (confirmación)**: Instantánea del código en un momento específico.

- **Rama:** Línea de desarrollo independiente para trabajar en nuevas funciones o correcciones.
- **Merge (fusión):** Combina cambios de una rama en otra.
- **Push:** Envía commits locales a un repositorio remoto.
- **Pull:** Trae cambios del repositorio remoto a tu copia local.

Flujo de trabajo típico

1. `git init` → inicializar un repositorio.
2. `git add` → preparar cambios para commit.
3. `git commit -m "mensaje"` → guardar cambios.
4. `git branch` → crear nuevas ramas (opcional).
5. `git checkout` → cambiar de rama (opcional).
6. `git merge` → fusionar ramas (opcional).
7. `git push` → enviar cambios a remoto.
8. `git pull` → obtener cambios de remoto.

Consejos para principiantes

- Aprende lo básico primero: commits, repositorios y seguimiento de cambios.
- Practica en un repositorio de prueba, creando ramas y fusionándolas.
- No dudes en buscar recursos en línea, tutoriales y foros.
- La **línea de comandos** ofrece más control que las interfaces gráficas.

Mentalidad Git

- Git no es solo una herramienta, sino una forma de pensar: estructurada, organizada y colaborativa.
- Fomenta **buenas prácticas de desarrollo**, experimentación segura y colaboración eficiente.

Qué es el control de versiones

- Es como una **máquina del tiempo para tu código**: permite rastrear todos los cambios realizados en un proyecto.
- Cada **instantánea del código** se llama **confirmación (commit)** y representa una versión específica del proyecto.
- Puedes **volver a una versión anterior** si cometes un error o deseas revisar cambios previos.

Beneficios principales

1. **Recuperación de errores**: puedes restaurar tu proyecto a un estado previo sin perder todo el trabajo.
2. **Colaboración**: varias personas pueden trabajar en el mismo proyecto, hacer cambios y crear commits sin conflictos.
3. **Experimentación segura**: puedes crear ramas (branches) para probar nuevas funciones; si algo falla, la rama principal permanece intacta.

Conceptos clave

- **Commit**: instantánea de tu proyecto en un momento específico, con un mensaje descriptivo.
- **Branch (rama)**: línea de desarrollo independiente para probar ideas o nuevas funcionalidades.
- **Merge (fusión)**: combinación de cambios de diferentes ramas en la principal.

Conclusión

- Aprender control de versiones es esencial para cualquier desarrollador de Python.
- Facilita escribir código más limpio, colaborar eficientemente y experimentar sin riesgos.
- La práctica constante aumenta la comodidad y confianza al usar estas herramientas.

Configuración de Git

- **Git** es un sistema de control de versiones distribuido (DVCS) que permite a cada desarrollador tener una copia completa del repositorio y su historial.
- Diferencia clave con sistemas centralizados: **trabajo offline, colaboración más flexible y mayor tolerancia a fallos.**

Requisitos previos

- Sistema operativo: Windows, macOS o Linux.
- Conexión a Internet: necesaria para clonar, enviar y sincronizar repositorios remotos.

Configuración inicial de Git

1. **Instalación:** descargar Git desde git-scm.com y verificar con `git --version`.

Configurar usuario global:

```
git config --global user.name "Your Name"
```

```
git config --global user.email "your_email@example.com"
```

2. Esto identifica al autor en cada commit.

Crear y gestionar un repositorio

Inicializar repositorio:

```
git init
```

-

Preparar cambios:

```
git add <filename>    # preparar un archivo
```

```
git add .              # preparar todos los cambios
```

-

Confirmar cambios:

```
git commit -m "Mensaje descriptivo"
```

-

Ramas y fusión

Crear rama:

```
git branch <branch_name>
```

-

Cambiar de rama:

```
git checkout <branch_name>
```

-

Fusionar rama en la principal:

```
git checkout main
```

```
git merge <branch_name>
```

-
- Resolver conflictos manualmente si es necesario.

Repositorios remotos

Clonar repositorio remoto:

```
git clone <repository_url>
```

-

Enviar cambios (push):

```
git push
```

-

Extraer cambios (pull):

```
git pull
```

-

Escenarios prácticos

1. **Colaboración en equipo:** cada miembro trabaja en su rama, luego se fusionan cambios en la principal sin conflictos.
2. **Experimentación:** crear ramas para probar nuevas funcionalidades sin afectar el proyecto principal.
3. **Recuperar versiones anteriores:** revertir a commits anteriores si se introduce un error, como una “máquina del tiempo” para el código.

Beneficios y consejos

- Git facilita **colaboración, mantenimiento del código y control del historial**.
- La curva de aprendizaje inicial puede ser empinada, pero los recursos en línea y la práctica hacen que sea accesible.
- Configurar Git correctamente es el primer paso para **gestionar proyectos de manera eficiente y segura**.

Configuración en GitHub (Crear una cuenta en GitHub)

Qué es GitHub

- GitHub es una **plataforma online para almacenar, compartir y colaborar en proyectos de código**.
- Funciona como una “red social para desarrolladores”, permitiendo proteger tu trabajo y colaborar globalmente.

Repositorios

- Un **repositorio** (repo) es como una carpeta online que contiene todos tus archivos, código, imágenes y el historial de cambios de tu proyecto.
- Se pueden crear repositorios **públicos** (visibles para todos) o **privados** (accesibles solo para ti o quienes invites).

- Es recomendable incluir un **archivo README**, que describe el proyecto, cómo usarlo y quién contribuye.

Crear un repositorio en GitHub

1. Inicia sesión en GitHub.
2. Haz clic en **Nuevo repositorio**.
3. Asigna un nombre, descripción y define si será público o privado.
4. Añade opcionalmente un README.
5. El repositorio queda listo para almacenar y rastrear tus archivos.

Añadir archivos y commits

- Para añadir un archivo:
 - Haz clic en **Añadir archivo** → **Crear nuevo archivo**.

Escribe el código, por ejemplo `Hello.py`:

```
print("¡Hola, GitHub!")
```

- - Cada archivo añadido requiere un **commit**, que es una instantánea del proyecto en ese momento.
 - Cada commit necesita un mensaje descriptivo para registrar los cambios realizados.

Funciones importantes de GitHub

- **Ramas (branches):** Permiten trabajar en nuevas funciones o correcciones sin afectar la rama principal.
- **Fusiones (merge):** Integran cambios de ramas al proyecto principal.
- **Pull requests:** Permiten que otros revisen y comenten tu código antes de fusionarlo.
- **Herramientas de gestión:** Seguimiento de problemas, gestión de proyectos y revisión de código.

- **Integraciones:** GitHub Marketplace permite conectarse con otras herramientas y servicios para mejorar tu flujo de trabajo.

Importancia de GitHub

- Protege tu código y asegura que siempre esté respaldado.
- Facilita la colaboración global y el trabajo en equipo.
- Es útil tanto para grandes proyectos como para desarrolladores individuales o pequeños equipos.

Diferencia entre Git y GitHub (Te enseñan a subir un archivo Python de Git a Github)

- **Git:** sistema distribuido de control de versiones que guarda cambios en tu código localmente, permite volver a versiones anteriores y trabajar con **ramas** para experimentar sin afectar al proyecto principal.
- **GitHub:** plataforma online basada en Git donde puedes almacenar, compartir y colaborar en proyectos desde cualquier lugar. Funciona como un “espacio social” para tu código.

Flujo básico con Git

Inicializar repositorio local

```
git init
```

1. → Git comienza a rastrear tu proyecto.

Preparar archivos para commit

```
git add .
```

2. → Indica qué archivos incluir en la próxima instantánea.

Hacer un commit

```
git commit -m "initial commit"
```

3. → Guarda una versión de tu proyecto en ese momento.

Crear y cambiar de rama

```
git checkout -b feature/new-greeting
```

4. → Permite trabajar en nuevas funciones sin afectar el código principal.

Fusionar ramas

```
git checkout main
```

```
git merge feature/new-greeting
```

5. → Integra cambios de la rama de trabajo a la rama principal.

Conexión con GitHub

1. **Crear un repositorio remoto** en GitHub (ejemplo: [MyPythonProject](#)).

Vincular repositorio local con el remoto:

```
git remote add origin <URL-repositorio>
```

- 2.

Enviar cambios al repositorio remoto:

```
git push -u origin main
```

3. → Tu código queda respaldado en GitHub.

Clonar repositorio desde GitHub (para obtener una copia completa en tu máquina):

```
git clone <URL-repositorio>
```


4.

Mantener sincronización con cambios de otros:

```
git pull
```

5.

Ideas clave

- Git = motor de control de versiones en tu máquina.
- GitHub = plataforma online para compartir y colaborar.
- Usarlos juntos garantiza que tu código esté **seguro, versionado y listo para trabajo en equipo**.

Cuestionario

1. Estás explicando a un nuevo miembro del equipo cómo funciona Git. Tienen curiosidad por conocer el mecanismo fundamental que emplea Git para controlar las modificaciones realizadas en los archivos de un proyecto. ¿Cuál es el concepto básico que les explicarías? Selecciona la mejor respuesta.

- ☐ Deltas
- ☒ Instantáneas
- ☐ Puntos de control
- ☐ Compromete

✔ **Correcto**

Correcto Git utiliza instantáneas para realizar un seguimiento de los cambios en los archivos, capturando el estado del proyecto en puntos específicos en el tiempo.

2. En el contexto de un repositorio Git, ¿cuál es la función principal de una confirmación? Selecciona la mejor respuesta.

- ☐ Pasar de una versión del proyecto a otra
- ☐ Crear una copia de seguridad del repositorio
- ☒ Guardar una instantánea del estado actual del proyecto
- ☐ Fusión de cambios de varias ramas

✔ **Correcto**

Correcto Un commit guarda una instantánea del estado actual del proyecto, lo que permite a los desarrolladores mantener un registro histórico de los cambios.

3. Estás a punto de embarcarte en un nuevo proyecto de programación y quieres configurar el control de versiones mediante Git. ¿Qué comando ejecutarías en tu terminal para crear un nuevo repositorio Git dentro del directorio actual de tu proyecto? Selecciona la mejor respuesta.

- ☒ git init
- ☐ git start
- ☐ git crear
- ☐ git nuevo

✓ **Correcto**

¡Correcto! Este comando inicializa un nuevo repositorio Git en tu directorio actual.

4. Has hecho algunos cambios en los archivos de tu proyecto, y estás listo para prepararlos para incluirlos en tu próximo commit. ¿Qué comando Git utilizarás para marcar estos cambios para la próxima confirmación? Selecciona la mejor respuesta.

- ☒ git add
- ☐ etapa git
- ☐ git push
- ☐ git commit

✓ **Correcto**

Correcto Este comando prepara los cambios para el siguiente commit.

5. ¿Cuál de los siguientes comandos se utiliza para enviar confirmaciones a un repositorio remoto en Git? Selecciona la mejor respuesta.

- ☐ git pull
- ☐ git clone
- ☐ git remoto
- ☒ git push

✓ **Correcto**

Correcto Este comando sube tus commits locales a un repositorio remoto.

Creación de una cartera de desarrollo de software

Importancia del portafolio

- Es más que código: muestra **habilidades, experiencia y personalidad**.
- Sirve como prueba tangible de tu capacidad para escribir código limpio y resolver problemas.
- Te diferencia en un mercado competitivo y genera conversación en entrevistas.

Currículum vs Portafolio

- **Currículum**: visión rápida y resumida de habilidades, experiencia y formación.
- **Portafolio**: muestra en detalle proyectos, código y logros.
- **Ambos son necesarios**: el CV filtra rápido, el portafolio profundiza en tus capacidades.

Qué incluir en un portafolio

- **Proyectos personales**: variedad de tecnologías y resolución de problemas reales.
- **Muestras de código**: limpias, documentadas y fáciles de entender.
- **Contribuciones a proyectos open source**: demuestran colaboración.
- **Estudios de caso**: retos, soluciones y resultados.
- **Redacción técnica**: blogs, documentación o tutoriales.

Qué evitar: proyectos irrelevantes, código desordenado, aportes mínimos o código confidencial.

Buenas prácticas

- Elige una **plataforma adecuada** (GitHub, GitLab o web personal).
- Prioriza **calidad sobre cantidad**.
- Destaca tecnologías y habilidades usadas en cada proyecto.
- Organiza el portafolio con **navegación clara**.
- Mantenlo **actualizado y vivo**.

Contar historias con tus proyectos

Cada proyecto debe narrar:

1. **El reto** (problema a resolver).
2. **Tu enfoque** (tecnologías y decisiones).
3. **El resultado** (impacto y aprendizajes).

Cómo presentarlo

- Muestra **entusiasmo y pasión** por programar.
- Prepara un **discurso breve** sobre ti y tu portafolio.
- Resalta lo que te diferencia.
- Practica respuestas a preguntas sobre proyectos y resolución de problemas.

Conclusión: Un portafolio sólido, bien narrado y actualizado es clave para destacar como desarrollador. Refleja no solo tus habilidades técnicas, sino también tu evolución, motivación y capacidad de aportar valor a un equipo.

Examinar una cartera/portfolio en GitHub

- Funciona como tu **currículum en línea** para mostrar proyectos y habilidades.
- Un perfil bien estructurado con **foto profesional, biografía breve y repositorios anclados** transmite profesionalismo.

Elementos clave de un perfil sólido

- **Repositorios anclados:** muestran tu mejor trabajo de forma visible.
- **Nombres claros y descriptivos** en los repositorios facilitan la comprensión.
- **Archivo README completo:** debe incluir título, descripción, tecnologías usadas, pasos de instalación y ejemplos.
- **Historial de commits coherente:** refleja constancia y evolución.
- **Uso de incidencias (issues):** muestra que gestionas errores y mejoras.

Ejemplos efectivos

- Perfiles con proyectos de **aprendizaje automático** o **Jupyter Notebooks** destacan porque combinan código, explicaciones y visualizaciones.
- Uso de **GitHub Pages** para crear un sitio personal con enlaces a proyectos, CV y blog.

Buenas prácticas al crear un repositorio

1. **Nombre claro** (ej. *Proyectos Python* en lugar de *MyCode*).
2. **Descripción breve** del contenido.
3. **Elegir licencia** (ej. MIT, común en proyectos open source).

4. **README estructurado en Markdown** con:

- título y descripción
- tecnologías utilizadas
- instrucciones de instalación
- ejemplos de uso
- imágenes o GIFs opcionales.

5. **Commits claros** con mensajes descriptivos.

6. **Empujar (push) cambios** al repositorio remoto en GitHub.

Construcción y mejora continua

- **Anclar el nuevo repositorio** en tu perfil para darle visibilidad.
- Actualizar y agregar proyectos conforme aprendes.
- Inspirarte explorando otros perfiles en **GitHub Explorer**.
- Usar **GitHub Skills** para practicar y aprender.

Conclusión: Git rastrea los cambios en el código, mientras que GitHub los aloja y facilita la colaboración. Un buen portafolio en GitHub debe estar cuidado, actualizado y mostrar lo mejor de tus proyectos y habilidades.

Configurar tu cuenta de GitHub y crear un portafolio profesional:

GitHub: más que almacenamiento de código

- Es la **red social del código**, donde los desarrolladores comparten proyectos, colaboran y muestran sus habilidades.
- Un perfil en GitHub bien cuidado es una **ventaja clave para programadores de Python**, ya que muestra evidencia práctica de conocimientos, no solo una lista en un currículum.

Beneficios de usar GitHub para tu portafolio

1. Demuestra habilidades reales:

- Repositorios con proyectos (ej. *Machine Learning con scikit-learn* o *apps web con Flask*).

2. Destaca pasión y experiencia:

- Proyectos personales y contribuciones a código abierto reflejan iniciativa.

3. Colaboración y comunidad:

- Participar en proyectos de código abierto demuestra trabajo en equipo, estándares de calidad y compromiso con la comunidad.

Ventajas y desventajas

Ventajas:

- Visibilidad ante reclutadores.
- Colaboración sencilla en proyectos.
- Control de versiones con Git.
- Comunidad activa para aprender y hacer networking.

Desventajas:

- Exposición pública del código (aunque se pueden usar repos privados).

- Curva de aprendizaje de Git/GitHub.

Configuración básica de una cuenta de GitHub

1. Registrarse en **github.com** con un nombre de usuario profesional.
2. Elegir plan (gratuito suele bastar).
3. Verificar correo electrónico.
4. Personalizar el perfil: foto profesional, bio breve y enlaces relevantes.

Estrategias para un perfil profesional

- **Curar repos públicos:** mostrar solo los mejores proyectos, archivar/privatizar lo irrelevante.
- **README de perfil:** breve presentación con habilidades, experiencia y enlaces a proyectos.
- **GitHub Pages:** crear un sitio personal/portafolio.
- **Contribuir a proyectos de código abierto:** refuerza habilidades técnicas y colaboración.
- **Usar un nombre y correo profesional.**
- **Cuidar tu feed de actividad** para transmitir profesionalismo.
- **Organizaciones:** separar proyectos personales y profesionales.

Aprovechar las funciones de GitHub

- **Repositorios anclados:** destacar proyectos clave.
- **Tableros de proyectos:** mostrar gestión de tareas.

- **Gráfico de contribuciones:** evidencia de constancia.

Crear un portafolio en GitHub: pasos

1. Seleccionar proyectos variados (web, data, automatización, ML).
2. Crear repositorios con nombres claros y README detallados.
3. Mantener código ordenado, comentado y legible.
4. Escribir mensajes de commit claros.
5. Destacar tecnologías y habilidades usadas.

Buenas prácticas

- Contribuir a proyectos de código abierto.
- Participar en debates y dar feedback en la comunidad.
- Usar GitHub Pages para un portafolio visual.
- Mantener perfil y repositorios **actualizados**.

Conclusión

Un buen perfil de GitHub es más que código: refleja tus **habilidades técnicas, colaboración, profesionalismo y pasión por aprender**. Es la primera impresión que verán muchos empleadores, por lo que debe estar **bien cuidado, actualizado y mostrar lo mejor de ti como programador de Python**.

Cuestionario

1. Estás seleccionando proyectos para incluirlos en tu carpeta. Tienes una mezcla de proyectos personales y de proyectos realizados para un curso académico. ¿Qué tipo de proyecto es más valioso para incluir en un portafolio profesional? Selecciona la mejor respuesta.

- ☐ Proyectos teóricos basados en escenarios hipotéticos.
- ☒ Proyectos que resuelvan problemas del mundo real o demuestren aplicaciones prácticas.
- ☐ Proyectos inacabados con código complejo pero sin una funcionalidad clara.
- ☐ Ejercicios sencillos de codificación a partir de tutoriales introductorios.

✔ **Correcto**

Correcto Los proyectos con relevancia en el mundo real demuestran tu capacidad para aplicar tus conocimientos.

2. Estás decidiendo qué tipos de archivos incluir en tu repositorio de GitHub para un proyecto de Python. ¿Cuál de los siguientes tipos de archivos se incluye normalmente en un repositorio de GitHub para un proyecto Python? Selecciona todos los que correspondan.

- ☐ Archivos ejecutables como .exe o .app.
- ☒ archivos .py que contienen código Python.

✔ **Correcto**

¡Correcto! los archivos .py son esenciales ya que contienen el núcleo lógico de tu proyecto Python.

- ☒ Archivos de datos utilizados por sus scripts Python.

✔ **Correcto**

Correcto Los archivos de datos suelen ser necesarios para que su proyecto funcione correctamente.

- ☒ Archivo README.md con una descripción del proyecto.

✔ **Correcto**

Correcto Un archivo README es crucial para explicar tu proyecto a los demás.

3. Estás creando un portafolio para mostrar tus habilidades en Python y quieres asegurarte de que está bien estructurado y es informativo. ¿Cuáles de los siguientes elementos deben incluirse en un portafolio profesional para un desarrollador de Python? Seleccione todos los que corresponda.

- ☐ Anécdotas e historias personales no relacionadas con su experiencia profesional.
- ☒ Información de contacto para que los posibles empleadores puedan ponerse fácilmente en contacto con usted.

✔ **Correcto**

Correcto Facilitar el contacto es esencial para las oportunidades potenciales.

- ☒ Descripciones de proyectos con explicaciones claras de su papel y contribuciones.

✔ **Correcto**

Correcto Contextualizar sus proyectos ayuda a los espectadores a entender su implicación.

- ☒ Enlaces a sus repositorios de GitHub u otras plataformas de alojamiento de código.

✔ **Correcto**

Correcto Facilitar el acceso a su código permite una evaluación más profunda de sus competencias.

4. Estás trabajando en un proyecto complejo de Python con múltiples archivos y funciones. Quieres organizar tu código de forma efectiva. ¿Qué práctica es la más útil para mantener tu código organizado y fácil de entender? Selecciona la mejor respuesta.

- ☐ Escribir todo el código en un único archivo largo sin funciones ni módulos.
- ☒ Descomponer el código en funciones más pequeñas y reutilizables con nombres y objetivos claros.
- ☐ Utilizar nombres de variables muy cortos y comentarios crípticos para ahorrar espacio.
- ☐ Evitar el uso de comentarios, asumiendo que el código se explica por sí mismo.

✔ **Correcto**

Correcto El código modular es más fácil de gestionar y depurar.

5. Imagina que eres un jefe de contratación que está revisando solicitudes para un puesto de desarrollador junior de Python. Dos candidatos tienen currículos similares en los que se enumeran cursos relevantes y habilidades básicas. Sin embargo, uno de ellos ha incluido un enlace a un portafolio bien organizado con proyectos personales y muestras de código, mientras que el otro no. ¿Cuál es la ventaja MÁS probable que tiene el candidato con el portafolio en esta situación? Seleccione la mejor respuesta.

- ☐ Es probable que el candidato sin cartera tenga menos educación formal y certificaciones.
- ☐ Es probable que el candidato sin cartera se haya centrado menos en la creación de redes y tenga menos contactos en el sector.
- ☐ El candidato con la cartera posee una mayor confianza en sus habilidades de codificación, lo que se traducirá en un mejor rendimiento en las entrevistas.
- ☒ El candidato con la cartera ha proporcionado ejemplos concretos de sus habilidades de codificación.

✔ **Correcto**

Correcto El portafolio ofrece pruebas tangibles de sus habilidades más allá de lo que puede transmitir un currículum.

Resolver problemas con Python:

Python y la depuración

- Python es versátil y legible, ideal para web, ciencia de datos, automatización y ML.

- En la práctica, los datos son **desordenados e impredecibles**, lo que causa errores y bloqueos.
- La depuración es esencial: permite encontrar y corregir errores, asegurando programas fiables.

Problemas comunes con datos desordenados

- **Entradas de usuario incorrectas** (ej. texto en lugar de números).
- **Cambios en fuentes externas** (páginas web rediseñadas, bases de datos obsoletas).
- **Archivos inconsistentes** (CSV con comas extra, valores faltantes, columnas desordenadas).

Técnicas esenciales de depuración

1. **print()** → inspeccionar valores en la ejecución.
2. **Mensajes de error** → pistas claras de dónde ocurre el problema.
3. **Depurador (pdb)** → recorrer el código paso a paso.
4. **Registro (logging)** → crear un historial detallado de la ejecución.
5. **Pruebas unitarias** → detectar errores introducidos por cambios en el código.

Ejemplo práctico: limpiar un CSV desordenado

Proceso paso a paso:

1. **Examinar** el archivo → detectar valores faltantes, incoherencias y errores de tipo.
2. **Rellenar o eliminar datos faltantes** → usar valores por defecto o descartar filas incompletas.

3. **Normalizar** → estandarizar cadenas (ej. nombres en mayúsculas/minúsculas).
4. **Manejar valores incorrectos** → usar `try-except`, convertir o registrar problemas.

Resultado: un conjunto de datos limpio, listo para análisis confiable.

La mentalidad de la depuración

- Depurar = más que corregir errores → es **resolver problemas con curiosidad, paciencia y persistencia**.
- Cada error es una oportunidad de aprendizaje.
- La comunidad Python es un recurso valioso: preguntar, compartir y aprender.

Conclusión

- Dominar la depuración convierte a un programador en un **Pythonista más seguro, resiliente e ingenioso**.
- La depuración es clave para construir aplicaciones **robustas, adaptables y fiables** en entornos reales.

Actividad FINAL

1. Estás desarrollando una función para calcular impuestos. ¿Cómo ayudarían las pruebas unitarias a garantizar su exactitud? Seleccione la mejor respuesta.

- ☐ Comparando la salida de la función con los resultados de otro software de cálculo de impuestos.
- ☐ Pidiendo a los usuarios que prueben la función e informen de cualquier error que encuentren.
- ☒ Comprobando si la salida de la función coincide con los resultados esperados para varios niveles de ingresos y tramos impositivos.
- ☐ Generando automáticamente formularios fiscales para distintos escenarios.

✔ **Correcto**

Correcto Las pruebas unitarias proporcionarían diferentes escenarios de ingresos a la función y verificarían si los impuestos calculados son correctos, de forma similar a la prueba de engranajes individuales en una máquina antes del montaje.

2. Tienes una función que calcula el área de un círculo. ¿Cuál de los siguientes es un ejemplo de aserción que podrías utilizar en una prueba unitaria para esta función? Seleccione la mejor respuesta.

- ☒ `assert calcular_área(5) == 78.53981633974483`
- ☐ `assert type(calcular_área(3)) == float`
- ☐ `assert calcular_área(0) no es None`
- ☐ `assert calcular_área(-2) produce ValueError`

✔ **Correcto**

Correcto Esta aserción comprueba si el área calculada para un círculo de radio 5 coincide con el valor esperado.

3. Has hecho algunos cambios en tu código y quieres crear una instantánea de esos cambios en tu repositorio Git. ¿Qué comando utilizarías? Selecciona la mejor respuesta.

- ☐ `git init`
- ☐ `git add`
- ☐ `git push`
- ☒ `git commit`

✔ **Correcto**

Correcto El comando `git commit` crea una instantánea de tu proyecto en un punto específico en el tiempo, capturando todos los cambios que has hecho desde el commit anterior.

4. Ha realizado cambios en varios archivos de su proyecto y desea incluirlos en su próxima confirmación. ¿Qué comandos utilizarías para conseguirlo? Selecciona todos los que correspondan.

- ☐ git commit
- ☐ git push
- ☒ git add <nombre de archivo>

✔ **Correcto**

Correcto El comando git add <nombredearchivo> se utiliza para preparar archivos específicos, preparándolos para ser incluidos en la siguiente confirmación.

- ☒ git add

✔ **Correcto**

Correcto El comando git add . escenifica todos los cambios en el directorio actual y sus subdirectorios, preparándolos para ser incluidos en su próxima confirmación.

5. Estás trabajando en un proyecto complejo de Python y quieres asegurarte de que puedes realizar un seguimiento de los cambios, experimentar con nuevas funciones sin romper el código existente y colaborar eficazmente con tu equipo. ¿Qué concepto sería más beneficioso para lograr estos objetivos? Selecciona la mejor respuesta.

- ☐ Estructuras de los datos y algoritmos
- ☒ Control de versión
- ☐ Programación orientada a objetos
- ☐ Aprendizaje automático

✔ **Correcto**

Correcto Los sistemas de Control de versión, como Git, proporcionan las funciones esenciales para hacer un seguimiento de los cambios, experimentar con seguridad y colaborar eficazmente en proyectos de software.

6. En un proyecto colaborativo de desarrollo de software, ¿cuál de las siguientes situaciones pone de manifiesto la importancia del control de versiones? Seleccione todas las que procedan.

☒ Varios desarrolladores trabajan simultáneamente en distintas funciones.

☒ **Correcto**

Correcto El control de versión ayuda a gestionar y fusionar los cambios de varios desarrolladores que trabajan en paralelo.

☒ Necesita volver a una versión anterior del código debido a un error o una consecuencia no deseada.

☒ **Correcto**

Correcto Control de versión le permite volver fácilmente a estados anteriores de la base de código si es necesario.

☐ Desea automatizar el despliegue de su código en un entorno de producción.

☒ Es necesario seguir el historial de cambios realizados en el código base para comprender su evolución.

☒ **Correcto**

Correcto El Control de versión mantiene un historial detallado de todos los cambios, lo que facilita el seguimiento de la evolución del código base.

7. ¿Cuál de las siguientes funciones de pytest le ayuda a comprobar si su código produce los resultados esperados? Seleccione la mejor respuesta.

- ☐ Organización de las pruebas
- ☐ Parametrización
- ☐ Accesorios
- ☒ Afirmaciones

aserciones

✓ **Correcto**

Correcto Las aserciones se utilizan para verificar que el código se comporta como se espera.

8. Estás escribiendo una prueba para una función `divide(a, b)` que divide dos números. ¿Cómo escribirías una prueba pytest para comprobar si se produce un `ZeroDivisionError` cuando `b` es cero? Seleccione la mejor respuesta.

- ☐ `assert divide(10, 0) is None`
- ☐ `try: divide(10, 0) except ZeroDivisionError: pass`
- ☐ `assert divide(10, 0) == ZeroDivisionError`
- ☒ `with pytest.raises(ZeroDivisionError): divide(10, 0)`

✓ **Correcto**

Correcto El gestor de contexto `pytest.raises`` es la forma adecuada de comprobar las excepciones esperadas.

8. Estás escribiendo una prueba para una función `divide(a, b)` que divide dos números. ¿Cómo escribirías una prueba `pytest` para comprobar si se produce un `ZeroDivisionError` cuando `b` es cero? Seleccione la mejor respuesta.

- ☐ `assert divide(10, 0) is None`
- ☐ `try: divide(10, 0) except ZeroDivisionError: pass`
- ☐ `assert divide(10, 0) == ZeroDivisionError`
- ☒ `con pytest.raises(ZeroDivisionError): divide(10, 0)`

✓ **Correcto**

Correcto El gestor de contexto `pytest.raises`` es la forma adecuada de comprobar las excepciones esperadas.

9. Usted tiene múltiples funciones de prueba que requieren una configuración específica, tales como el establecimiento de variables de entorno o la inicialización de una biblioteca con ciertos parámetros. ¿Cuál es la mejor manera de evitar repetir este código de configuración en cada función de prueba? Seleccione la mejor respuesta.

- ☒ Crea un fixture que realice la configuración y utilízalo en tus funciones de prueba.
- ☐ Utilice los marcadores de `pytest` para agrupar las pruebas y aplicar el código de configuración a todo el grupo.
- ☐ Defina una clase base con el código de configuración y haga que sus clases de prueba hereden de ella.
- ☐ Copie y pegue el código de configuración en cada función de prueba.

✓ **Correcto**

Correcto Los Fixtures permiten definir código de configuración reutilizable, promoviendo la eficiencia y la mantenibilidad.

10. Ha escrito una función que realiza un cálculo complejo. Quiere probarla con una amplia gama de valores de entrada válidos e inválidos para asegurarse de que maneja todos los escenarios con gracia. ¿Cuál es la forma más eficiente de hacer esto en `pytest`? Seleccione la mejor respuesta.

- ☐ Pruebe manualmente la función con diferentes valores de entrada y verifique los resultados.
- ☒ Utilice la función de parametrización de `pytest` para ejecutar la misma función de prueba con diferentes valores de entrada.
- ☐ Escriba una función de prueba distinta para cada valor de entrada.
- ☐ Utilice otro marco de pruebas especializado en cálculos numéricos.

✓ **Correcto**

Correcto La parametrización le permite probar una función con múltiples entradas sin escribir funciones de prueba redundantes.

RESUMEN COMPLETO DEL CURSO

Logro general

Has completado con éxito el curso y adquirido una base sólida en Python, pasando de principiante a alguien capaz de escribir scripts, manipular datos y crear aplicaciones sencillas. Estás preparado para proyectos más complejos y continuar tu camino hacia el desarrollo experto en Python.

Módulo 1: Introducción a Python

- Conociste Python como un lenguaje potente, claro y versátil.
- Configuraste tu entorno de desarrollo.
- Diferencias entre scripting y programación.
- Escribiste y ejecutaste tus primeros programas.

Módulo 2: Conceptos básicos de Python

- Tipos de datos: enteros, flotantes, cadenas.
- Operadores y variables para manipular datos.
- Flujo de control: condicionales (`if-else`) y bucles (`for`, `while`).
- Capacidad de escribir programas dinámicos que toman decisiones y realizan cálculos.

Módulo 3: Funciones y módulos

- Funciones: bloques reutilizables de código que resuelven tareas específicas.
- Módulos: colecciones de funciones y variables que pueden importarse.
- Diferencia entre módulos incorporados y librerías externas.

- Organización del código, reducción de repetición y mejora de eficiencia.

Módulo 4: Estructuras de datos en Python

- Listas, tuplas, diccionarios y conjuntos: cómo almacenar y organizar datos.
- Operaciones básicas: acceso, modificación, iteración.
- Algoritmos básicos: ordenación y búsqueda.
- Elección de la estructura adecuada según la tarea.

Módulo 5: Manejo de errores y depuración

- Identificación de errores comunes y excepciones de Python.
- Uso de `try-except` para manejar errores y evitar que los programas se bloqueen.
- Técnicas y herramientas de depuración para localizar problemas.
- Escritura de código robusto y seguro.

Módulo 6: Fundamentos de pruebas y control de versiones

- **Pruebas con pytest:** escribir pruebas unitarias para verificar que partes del código funcionan correctamente.
- **Git:** sistema de control de versiones distribuido.
 - Rastrear cambios, revertir a versiones anteriores.
 - Trabajar de manera colaborativa mediante ramas y fusiones.
 - Gestión eficiente del historial del proyecto y colaboración segura.

Conexión entre módulos

- Combinar estructuras de datos, flujo de control, funciones, módulos y depuración permite escribir scripts limpios y organizados.
- Las pruebas y Git aseguran código fiable y colaborativo.
- Estas habilidades forman la base para el crecimiento continuo como desarrollador de Python.

Próximos pasos

- Avanzar hacia **Análisis de datos y Aprendizaje automático** con Python.
- Aprender librerías como **Pandas** y **NumPy** para procesar y analizar datos.
- Crear visualizaciones para comunicar resultados de manera efectiva.
- Implementar modelos de aprendizaje automático básicos y generar conjuntos de datos sintéticos para pruebas y experimentación.

Clave del éxito

- La práctica constante es esencial.
- Escribir código, experimentar y explorar recursos en línea y comunidad.
- Aprovechar los conocimientos adquiridos para crear proyectos y consolidar habilidades.

En resumen, este curso te dio los fundamentos completos de Python: escritura de código limpio, manejo de datos, funciones y módulos, depuración, pruebas y control de versiones, preparándote para retos más avanzados en análisis de datos, desarrollo de aplicaciones y aprendizaje automático.